

CMSC411: Computer Systems Architecture
Project 1: Branch Prediction
Due: 03/06/2024, 11:59 pm, EST (via Gradescope)

Important policies

1. Sharing of code between students is viewed as cheating and will receive appropriate action in accordance with university policy.
2. It is acceptable for you to compare your results, and only your results, with other students to help debug your program. It is not acceptable to collaborate either on the code development or on the final experiments.
3. You should do all your work in the C or C++ programming language and should be written according to the C99 or C++11 standards, using only the standard libraries.

Project Description

In this project, we will be designing a simple simulator for predicting the outcomes of branches. Trace files (".trc" files in test-trace folder) are provided, with each line indicating the PC value of each branch, and the actual outcome (e.g., 0x10 T). The simulator will process these traces, predicts the outcome of each branch, and verifies the prediction.

You will be modeling a few different predictor types. The predictors also accept the following input parameters so that we can vary the configuration:

1. **The predictor type (ptype)** - You should implement the bimodal, two-level adaptive, local history (p-share), and g-share branch predictors.
2. **The number of entries in the predictor table (num_entries)** - We use the lower bits of an address as the hash to the table. For example, in a 4-entry table, address in 000 (in binary) should go to entry 0, 001 to entry 1, 010 to entry 2, and 011 to entry 3, and 100 to entry 0 again.
3. **Bits per counter (counter_bits)** - The counters are initialized as weakly taken. For example, for a 3-bit counter, the counter value is 0-7, and the initial value is 4.
4. **Bits for each history register (history_bits)** - The history register is initialized as all not-taken. For example, 2-bit history should be initialized as 00.

To follow the thought process involved in designing a branch predictor, we will also look at the tradeoffs between **storage** and **accuracy**; more accurate predictors tend to require more storage overhead and attempt to find the right balance.

Specification of simulator and what to do

Before going to the details of what you need to do, we briefly explain a general overview of the code. For this project, we provide you with following files and folders:

- branchsim.cpp: You need to make your changes in this file, which will include the implementations of a few branch predictors.

- `branchsim_driver.cpp`: This file includes the `main()` function and runs the simulator for you. You don't have to change anything in this file. But feel free to read it to better get familiar with the code.
- `branchsim.hpp`: This file includes the definition of the functions, structs, etc. You don't have to change anything in this file. But feel free to read it to better get familiar with the code.
- `Makefile`: This is the makefile that you need to run to compile your predictor:
 - `$make`
 - `$make clean`
- `/test-traces` and `/test-output`: These two folders respectively include a group of test traces that your simulator will read and the output that it must generate if your simulator works correctly.
- `/traces`: This folder includes three real-world benchmarks that you will use to optimize your branch predictors for.
- `validate-test-traces.sh`: You will be running this script to test your branch predictor. Basically, this script runs `branchsim` for all the traces in `/test-traces`, writes the outputs in `/myoutput`, compares everything in `/test-output` with `/myoutput` and prints out the differences.

1. **The 'setup_predictor' function:** This function initializes the branch predictor and calculates the storage overhead of each. In the three parts specified in this function with 1.1, 1.2, and 1.3 (see the following figure), you need to calculate the size of the two-level adaptive, local history, and g-share branch predictors, respectively, and update '`p_stats->storage_overhead`' with that. The code for bimodal branch predictor is provided as an example.

```

49     switch (ptype) {
50     case PTYPE_BIMODAL: {
51         pht_size = num_entries;
52         p_stats->storage_overhead = ((std::uint64_t)pht_size) * ((std::uint64_t)counter_bits);
53         break;
54     }
55     case PTYPE_TWO_LEVEL_ADAPTIVE: {
56         /*
57          * 1.1. Add your code here
58          */
59         break;
60     }
61     case PTYPE_LOCAL_HISTORY: {
62         /*
63          * 1.2. Add your code here
64          */
65         break;
66     }
67     case PTYPE_GSHARE: {
68         /*
69          * 1.3. Add your code here
70          */
71         break;
72     }
73     }

```

2. **The 'get_index' function:** This function returns the index, using which we look up the pattern history table (PHT). In the three parts specified in this function with 2.1, 2.2, and 2.3 (see the following figure), you need to calculate the pht index for bimodal, two-level adaptive, and g-share branch predictors, respectively, and return it. In this function, the code for local history branch predictor is provided as an example.

```

107 ~ switch (pred_type) {
108 ~     case PTYPE_BIMODAL: {
109 ~         /*
110 ~          * 2.1. Add your code here
111 ~          */
112 ~     }
113 ~     case PTYPE_TWO_LEVEL_ADAPTIVE: {
114 ~         /*
115 ~          * 2.2. Add your code here
116 ~          */
117 ~     }
118 ~     case PTYPE_LOCAL_HISTORY: {
119 ~         std::uint64_t hist_index = pc & index_mask;
120 ~         return (hist_index * (1 << history_shift)) + (hist_table[hist_index] & history_mask);
121 ~     }
122 ~     case PTYPE_GSHARE: {
123 ~         /*
124 ~          * 2.3. Add your code here
125 ~          */
126 ~     }
127 ~     default:
128 ~         return 0;
129 ~ }

```

3. **The ‘predict_branch’ function:** This function queries the branch predictor for branch located at pc. The prediction result is returned as an enum branch_dir, which can be either TAKEN ('T'), or NOT_TAKEN ('N'). You may update any branch-related stats in p_stats as needed. More specifically, this function includes three parts: first, it updates the total number of branches (i.e., p_stats->num_branches); then, it gets the index based on pc using the get_index function we defined above; finally, it predicts the branch by accessing the pht. You need to write the required code for the last part, specified with 3.1 in this function (see the figure below). The template code always returns ‘TAKEN’. You need to remove line 154 and replace it with your code.

```

142 branch_dir predict_branch(std::uint64_t pc, branch_stats_t* p_stats) {
143     // Increment branch count
144     p_stats->num_branches++;
145
146     // Identify index
147     std::uint64_t index = get_index(pc);
148
149     // Predict the branch by accessing the pht
150     /*
151     * 3.1. Add your code here
152     * The following code (i.e., return TAKEN) is added just as an example to return TAKEN by default.
153     */
154     return TAKEN;
155 }

```

4. **The ‘update_predictor’ function:** This function updates the branch predictor and trains it for the next query. You may update any branch-related stats in p_stats as needed in part specified with 4.1. and 4.2.

```

169 ~ void update_predictor(std::uint64_t pc,
170 ~                     branch_dir actual,
171 ~                     branch_dir predicted,
172 ~                     branch_stats_t* p_stats) {
173 ~     std::uint64_t index = get_index(pc);
174 ~
175 ~     if (actual == predicted) {
176 ~         p_stats->correct++;
177 ~     }
178 ~
179 ~     if (actual == TAKEN) {
180 ~         ghr = (ghr << 1) | 1;
181 ~         /*
182 ~          * 4.1. Add your code here
183 ~          */
184 ~     } else {
185 ~         ghr = (ghr << 1) | 0;
186 ~         /*
187 ~          * 4.2. Add your code here
188 ~          */
189 ~     }
190 ~ }

```

5. **The ‘complete_predictor’ function:** This function cleans up memory and calculates the overall system statistics. You do not need to change anything in this function.

Statistics (output)

The simulator outputs the following statistics after completion of the run:

1. number of branches
2. number of branches predicted to be taken
3. number of branches predicted to be not-taken
4. number of correct predictions
5. misprediction rate
6. storage overhead

Validation

We have provided several test traces in the [test-traces directory](#), along with the correct output in the [output directory](#). You must run your simulator and debug it until it matches 100% all the statistics in the validation outputs. The validation script, `validate-test-traces.sh`, will do all that for you. You just need to make it an executable, with

```
chmod +x ./validate-test-trace.sh
```

or run it with

```
sh ./validate-test-traces.sh
```

The script will run all the configuration and run a diff between validated output and your output and print out all the differences. If the script does not print anything, you are good to go to the next step. If it prints some diff, I suggest that you first find the configuration that causes that diff and then run that instance alone to debug it.

Run Some Experiments:

For each benchmark in the ‘/traces’ directory (bzip2, gcc, and h264ref), design a branch predictor for the following goals:

1. You have a total budget of 4096 bits for the entire predictor (including history and counters)
2. The predictor should have the lowest possible misprediction rate.

To do so, you may vary any parameter (ptype, num_entries, counter_bits, history_bits).

By designing, we do not mean to create a brand-new predictor, it means to vary the above parameters. The method you use to find the best predictor is your choice. You can do pure brute force, or you may come up with an intelligent method based on some insight. I will define a range based on the overall class performance, and if you are within that range, you will get the point for this part.

Deliverables

What to hand in via Gradescope:

- **branchsim.cpp**
 - The commented source code for the simulator program.
- **<UID>_prj1.pdf**
 - A document with the design results of the experiments for each trace file, with a persuasive argument of the choices that were made. An argument may be as simple as an explanation of the search procedure used to find the designs and a statement about why the procedure is complete. This argument should include output from runs of your program. There are multiple answers for each trace file, so we will know which students have "collaborated" inappropriately!

Remember that your code must compile and run on a current variant of Linux (i.e., Debian, Red Hat, Ubuntu) running on an x86 architecture (i.e., Intel or AMD).

Grading Rubric

0% You do not hand in anything
+36% (manual grading) Your simulator doesn't run, does not work, but you hand in significant commented code (depending on how many parts of the code you completed you gain maximum 36%)
+48% Your simulator matches validation outputs
+16% (manual grading) You ran all experiments and found the best-performing predictors
Important note: Late submissions will be deducted 25% per day